

▼ Tarea Académica - Machine Learning

Predicción de complicaciones fatales en infarto de miocardio

Dataset: Myocardial infarction complications - UCI Machine Learning Repository

Tarea: Clasificación binaria usando la variable `LET_IS`

Línea base a replicar: Ghafari et al. (2023), quienes comparan Logistic Regression, SVM, Random Forest y XGBoost usando selección de características con RFE.

En este notebook se implementa una versión preliminar/base para la primera entrega del proyecto. La idea es tener un flujo reproducible que permita:

1. Cargar el dataset desde UCI.
2. Explorar la estructura de los datos.
3. Construir la variable objetivo binaria a partir de `LET_IS`.
4. Preparar los datos con imputación y escalamiento.
5. Entrenar modelos base comparables con la literatura.
6. Evaluar métricas relevantes para clasificación médica.
7. Dejar una sección inicial para mejoras posteriores.

El informe preliminar describe este conjunto como un dataset clínico de 1700 instancias y 111 variables predictoras. Por ello, el notebook imprimirá las dimensiones reales al cargarlo desde UCI para validar que coincidan con la descripción del informe.

▼ 1. Instalación de librerías

Esta celda está pensada para ejecutarse en Google Colab.

Si se usa Jupyter local y ya se tienen las librerías instaladas, se puede omitir.

```
!pip install ucimlrepo xgboost imbalanced-learn -q
```

▼ 2. Importación de librerías

```
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from ucimlrepo import fetch_ucirepo

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_validate
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import RFE

from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    confusion_matrix,
    classification_report,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    matthews_corrcoef
)

from xgboost import XGBClassifier

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
```

3. Carga del dataset

El dataset se carga directamente desde UCI mediante `ucimlrepo`.

Según la documentación del dataset, las variables predictoras corresponden a información clínica y demográfica del paciente, mientras que las variables objetivo representan complicaciones posteriores al infarto de miocardio.

```
# Cargar dataset desde UCI
mi_dataset = fetch_ucirepo(id=579)

X = mi_dataset.data.features.copy()
y_targets = mi_dataset.data.targets.copy()

print("Dimensiones de X:", X.shape)
print("Dimensiones de targets:", y_targets.shape)

display(X.head())
display(y_targets.head())
```

Dimensiones de X: (1700, 111)
Dimensiones de targets: (1700, 12)

	AGE	SEX	INF_ANAM	STENOK_AN	FK_STENOK	IBS_POST	IBS_NASL	GB	SIM_GIPERT	DLIT_AG	...	NOT_NA_1_n	NOT_NA_2_n	NOT_N
0	77.0	1	2.0	1.0	1.0	2.0	NaN	3.0	0.0	7.0	...	0.0	0.0	
1	55.0	1	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	1.0	0.0	
2	52.0	1	0.0	0.0	0.0	2.0	NaN	2.0	0.0	2.0	...	3.0	2.0	
3	68.0	0	0.0	0.0	0.0	2.0	NaN	2.0	0.0	3.0	...	0.0	0.0	
4	60.0	1	0.0	0.0	0.0	2.0	NaN	3.0	0.0	7.0	...	0.0	0.0	

5 rows × 111 columns

	FIBR_PREDS	PREDIS_TAH	JELUD_TAH	FIBR_JELUD	A_V_BLOK	OTEK_LANC	RAZRIV	DRESSLER	ZSN	REC_IM	P_IM_STEN	LET_IS
0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	1	0	0	0
4	0	0	0	0	0	0	0	0	0	0	0	0

4. Revisión inicial del dataset

```
print("Columnas predictoras:")
print(X.columns.tolist())

print("\nColumnas objetivo:")
print(y_targets.columns.tolist())

print("\nTipos de datos de X:")
display(X.dtypes.value_counts())

print("\nResumen estadístico:")
display(X.describe().T.head(20))
```

Columnas predictoras:

['AGE', 'SEX', 'INF_ANAM', 'STENOK_AN', 'FK_STENOK', 'IBS_POST', 'IBS_NASL', 'GB', 'SIM_GIPERT', 'DLIT_AG', 'ZSN_A', 'nr_11']

Columnas objetivo:

['FIBR_PREDS', 'PREDS_TAH', 'JELUD_TAH', 'FIBR_JELUD', 'A_V_BLOK', 'OTEK_LANC', 'RAZRIV', 'DRESSLER', 'ZSN', 'REC_IM', 'P_IM']

Tipos de datos de X:

	count
float64	110
int64	1

dtype: int64

Resumen estadístico:

	count	mean	std	min	25%	50%	75%	max	
AGE	1692.0	61.856974	11.259936	26.0	54.0	63.0	70.0	92.0	
SEX	1700.0	0.626471	0.483883	0.0	0.0	1.0	1.0	1.0	
INF_ANAM	1696.0	0.554835	0.836801	0.0	0.0	0.0	1.0	3.0	
STENOK_AN	1594.0	2.316186	2.440586	0.0	0.0	1.0	5.0	6.0	
FK_STENOK	1627.0	1.205286	1.040814	0.0	0.0	2.0	2.0	4.0	
IBS_POST	1649.0	1.160703	0.801400	0.0	0.0	1.0	2.0	2.0	
IBS_NASL	72.0	0.375000	0.487520	0.0	0.0	0.0	1.0	1.0	
GB	1691.0	1.393258	1.088803	0.0	0.0	2.0	2.0	3.0	
SIM_GIPERT	1692.0	0.033688	0.180478	0.0	0.0	0.0	0.0	1.0	
DLIT_AG	1452.0	3.340220	3.098646	0.0	0.0	3.0	7.0	7.0	
ZSN_A	1646.0	0.194411	0.658722	0.0	0.0	0.0	0.0	4.0	
nr_11	1679.0	0.025015	0.156217	0.0	0.0	0.0	0.0	1.0	
nr_01	1679.0	0.002382	0.048766	0.0	0.0	0.0	0.0	1.0	
nr_02	1679.0	0.011316	0.105806	0.0	0.0	0.0	0.0	1.0	
nr_03	1679.0	0.020846	0.142910	0.0	0.0	0.0	0.0	1.0	
nr_04	1679.0	0.017272	0.130323	0.0	0.0	0.0	0.0	1.0	
nr_07	1679.0	0.000596	0.024405	0.0	0.0	0.0	0.0	1.0	
nr_08	1679.0	0.002382	0.048766	0.0	0.0	0.0	0.0	1.0	
np_01	1682.0	0.001189	0.034473	0.0	0.0	0.0	0.0	1.0	
np_04	1682.0	0.001784	0.042207	0.0	0.0	0.0	0.0	1.0	

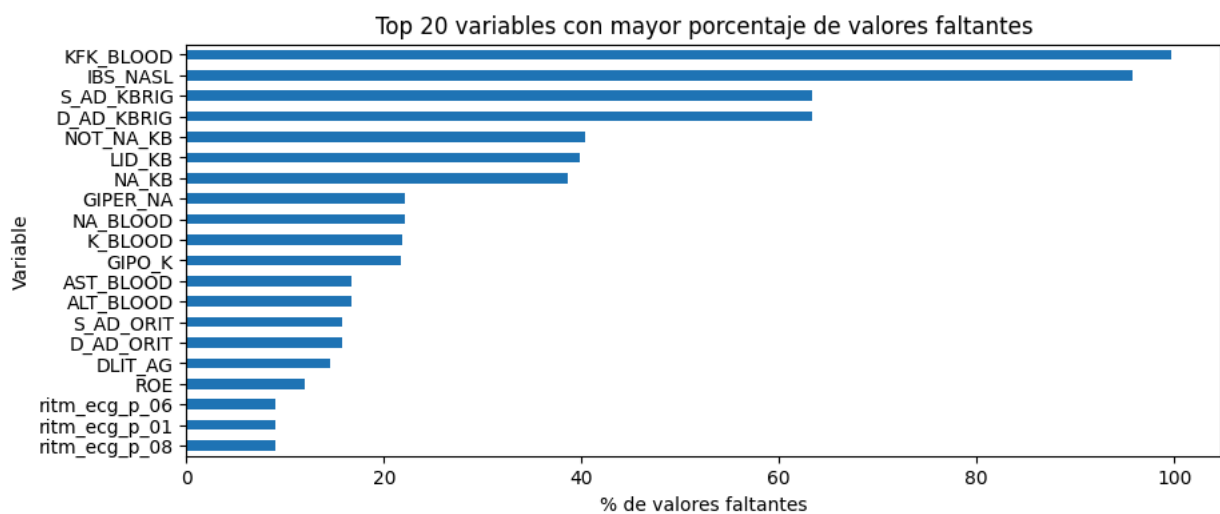
```
# Porcentaje de valores faltantes por variable
missing_percent = X.isna().mean().sort_values(ascending=False) * 100

print("Variables con más valores faltantes:")
display(missing_percent.head(20).to_frame("missing_%"))

plt.figure(figsize=(10, 4))
missing_percent.head(20).sort_values().plot(kind="barh")
plt.title("Top 20 variables con mayor porcentaje de valores faltantes")
plt.xlabel("% de valores faltantes")
plt.ylabel("Variable")
plt.show()
```

Variables con mas valores faltantes:

	missing_%
KFK_BLOOD	99.764706
IBS_NASL	95.764706
D_AD_KBRIG	63.294118
S_AD_KBRIG	63.294118
NOT_NA_KB	40.352941
LID_KB	39.823529
NA_KB	38.647059
NA_BLOOD	22.058824
GIPER_NA	22.058824
K_BLOOD	21.823529
GIPO_K	21.705882
AST_BLOOD	16.764706
ALT_BLOOD	16.705882
D_AD_ORIT	15.705882
S_AD_ORIT	15.705882
DLIT_AG	14.588235
ROE	11.941176
ritm_ecg_p_06	8.941176
ritm_ecg_p_08	8.941176
ritm_ecg_p_01	8.941176



5. Definición de la variable objetivo

Se utilizará la variable `LET_IS`, que indica el desenlace del paciente.

Para esta primera versión se transforma en una variable binaria:

- `0`: paciente vivo.
- `1`: paciente fallecido por alguna complicación.

En el dataset original, `LET_IS = 0` indica que el paciente sobrevivió, mientras que los valores `1` a `7` representan distintos tipos de desenlace fatal.

```
# Crear variable objetivo binaria
# 0 = vivo
# 1 = fallecido por complicación

target_col = "LET_IS"

if target_col not in y_targets.columns:
    raise ValueError(f"No se encontró la columna {target_col} en los targets.")

y = (y_targets[target_col] != 0).astype(int)
```

```
print("Distribución de la variable objetivo:")
display(y.value_counts().rename(index={0: "Vivo", 1: "Fallecido"}).to_frame("conteo"))

print("\nDistribución porcentual:")
display((y.value_counts(normalize=True) * 100).rename(index={0: "Vivo", 1: "Fallecido"}).to_frame("%"))

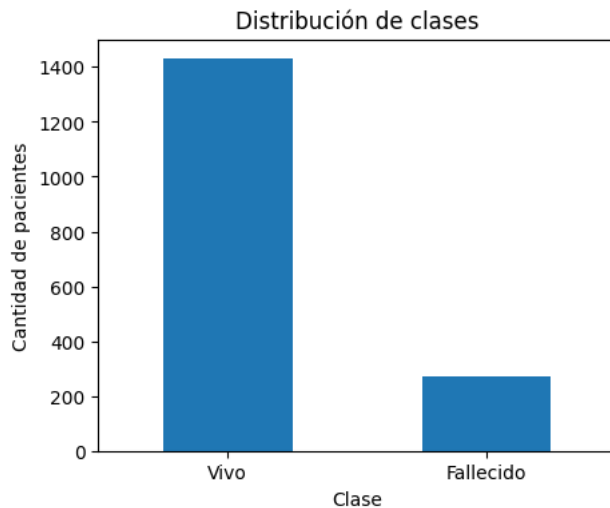
plt.figure(figsize=(5, 4))
y.value_counts().rename(index={0: "Vivo", 1: "Fallecido"}).plot(kind="bar")
plt.title("Distribución de clases")
plt.xlabel("Clase")
plt.ylabel("Cantidad de pacientes")
plt.xticks(rotation=0)
plt.show()
```

Distribución de la variable objetivo:

conteo	
LET_IS	
Vivo	1429
Fallecido	271

Distribución porcentual:

%	
LET_IS	
Vivo	84.058824
Fallecido	15.941176



6. Limpieza inicial de variables predictoras

Se elimina la columna **ID** si aparece dentro de las variables predictoras, ya que funciona como identificador del paciente y no debería usarse para entrenar el modelo.

```
# Eliminar identificadores si existen
cols_to_drop = [col for col in ["ID", "id"] if col in X.columns]

X_model = X.drop(columns=cols_to_drop, errors="ignore")

print("Columnas eliminadas:", cols_to_drop)
print("Dimensiones finales de X_model:", X_model.shape)
```

Columnas eliminadas: []
Dimensiones finales de X_model: (1700, 111)

7. Separación entrenamiento/prueba

Se usa una separación 80/20 y se aplica estratificación para mantener la proporción de clases en entrenamiento y prueba.

```
X_train, X_test, y_train, y_test = train_test_split(
    X_model,
    y,
    test_size=0.20,
    random_state=RANDOM_STATE,
```

```

    stratify=y
)

print("Tamaño de entrenamiento:", X_train.shape)
print("Tamaño de prueba:", X_test.shape)

print("\nDistribución en entrenamiento:")
display(y_train.value_counts(normalize=True).to_frame("proporción"))

print("\nDistribución en prueba:")
display(y_test.value_counts(normalize=True).to_frame("proporción"))

```

Tamaño de entrenamiento: (1360, 111)
 Tamaño de prueba: (340, 111)

Distribución en entrenamiento:

proporción 

LET_IS	
0	0.840441
1	0.159559

Distribución en prueba:

proporción

LET_IS	
0	0.841176
1	0.158824

8. Funciones de evaluación

En problemas clínicos no basta con mirar solo el `accuracy`.

También se evalúan:

- **Recall/Sensitivity:** capacidad para detectar correctamente pacientes fallecidos.
- **Specificity:** capacidad para detectar correctamente pacientes vivos.
- **Precision:** proporción de predicciones positivas correctas.
- **F1-score:** balance entre precisión y sensibilidad.
- **ROC-AUC:** capacidad general de separación entre clases.
- **MCC:** metrica robusta para datasets desbalanceados, incluida en el informe como metrica complementaria.

```

def compute_specificity(y_true, y_pred):
    """Calcula especificidad = TN / (TN + FP)."""
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    return tn / (tn + fp) if (tn + fp) > 0 else np.nan

def evaluate_model(model, X_test, y_test, model_name):
    """Evalua un modelo entrenado y devuelve metricas principales."""
    y_pred = model.predict(X_test)

    if hasattr(model, "predict_proba"):
        y_score = model.predict_proba(X_test)[:, 1]
    elif hasattr(model, "decision_function"):
        y_score = model.decision_function(X_test)
    else:
        y_score = None

    auc = roc_auc_score(y_test, y_score) if y_score is not None else np.nan

    metrics = {
        "Modelo": model_name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred, zero_division=0),
        "Recall/Sensitivity": recall_score(y_test, y_pred, zero_division=0),
        "Specificity": compute_specificity(y_test, y_pred),
        "F1-score": f1_score(y_test, y_pred, zero_division=0),
        "ROC-AUC": auc,
        "MCC": matthews_corrcoef(y_test, y_pred)
    }

    print("=" * 80)
    print(model_name)

```

```

print("=" * 80)
print("Matriz de confusion:")
print(confusion_matrix(y_test, y_pred))
print("\nReporte de clasificacion:")
print(classification_report(y_test, y_pred, zero_division=0))
print("\nMetricas principales:")
display(pd.DataFrame([metrics]))

return metrics

```

9. Experimento 1: modelos base sin selección de características

Este primer experimento sirve como punto de partida.

Se entrena cada modelo usando todas las variables disponibles, después de imputar valores faltantes.

Modelos incluidos:

1. Dummy Classifier.
2. Logistic Regression.
3. SVM con kernel RBF.
4. Random Forest.
5. XGBoost.

```

# Cálculo de scale_pos_weight para XGBoost por desbalance de clases
neg_count = (y_train == 0).sum()
pos_count = (y_train == 1).sum()
scale_pos_weight = neg_count / pos_count

print("Casos negativos en train:", neg_count)
print("Casos positivos en train:", pos_count)
print("scale_pos_weight:", scale_pos_weight)

```

```

Casos negativos en train: 1143
Casos positivos en train: 217
scale_pos_weight: 5.267281105990784

```

```

models_no_fs = {
    "Dummy Classifier": Pipeline(steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("model", DummyClassifier(strategy="most_frequent"))
    ]),

    "Logistic Regression": Pipeline(steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        ("model", LogisticRegression(
            max_iter=3000,
            class_weight="balanced",
            random_state=RANDOM_STATE
        ))
    ]),

    "SVM RBF": Pipeline(steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        ("model", SVC(
            kernel="rbf",
            probability=True,
            class_weight="balanced",
            random_state=RANDOM_STATE
        ))
    ]),

    "Random Forest": Pipeline(steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("model", RandomForestClassifier(
            n_estimators=300,
            class_weight="balanced",
            random_state=RANDOM_STATE,
            n_jobs=-1
        ))
    ]),

    "XGBoost": Pipeline(steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("model", XGBClassifier(
            n_estimators=300,
            learning_rate=0.05,

```

```
        max_depth=4,
        subsample=0.8,
        colsample_bytree=0.8,
        eval_metric="logloss",
        scale_pos_weight=scale_pos_weight,
        random_state=RANDOM_STATE
    ))
])
}
```

```
results_no_fs = []

for model_name, model in models_no_fs.items():
    model.fit(X_train, y_train)
    metrics = evaluate_model(model, X_test, y_test, model_name)
    metrics["Experimento"] = "Sin selección de características"
    results_no_fs.append(metrics)

df_results_no_fs = pd.DataFrame(results_no_fs)
display(df_results_no_fs.sort_values(by="ROC-AUC", ascending=False))
```



```

=====
Dummy Classifier
=====
Matriz de confusion:
[[286  0]
 [ 54  0]]

Reporte de clasificacion:
      precision    recall  f1-score   support

     0       0.84      1.00      0.91      286
     1       0.00      0.00      0.00       54

   accuracy      0.84      0.84      0.84      340
  macro avg       0.42      0.50      0.46      340
 weighted avg       0.71      0.84      0.77      340

Metricas principales:
=====

```

	Modelo	Accuracy	Precision	Recall/Sensitivity	Specificity	F1-score	ROC-AUC	MCC
0	Dummy Classifier	0.841176	0.0	0.0	1.0	0.0	0.5	0.0

```

=====
Logistic Regression
=====
Matriz de confusion:
[[225  61]
 [ 17  37]]

Reporte de clasificacion:
      precision    recall  f1-score   support

     0       0.93      0.79      0.85      286
     1       0.38      0.69      0.49       54

   accuracy      0.65      0.74      0.67      340
  macro avg       0.65      0.74      0.67      340
 weighted avg       0.65      0.74      0.67      340

```

10. Experimento 2: réplica preliminar con RFE

El estudio base de Ghafari et al. utiliza **Recursive Feature Elimination (RFE)** para reducir el conjunto de variables a 50 características.

En este experimento se replica esa idea de forma preliminar:

1. Imputación de valores faltantes.
2. Escalamiento.
3. Selección de 50 variables mediante RFE.
4. Entrenamiento de los modelos principales.
5. Evaluación con las mismas métricas.

Nota: esta es una réplica aproximada, ya que el paper puede tener detalles específicos de partición, configuración o preprocesamiento que no siempre están completamente disponibles.

```

Reporte de clasificacion:
      precision    recall  f1-score   support

# Estimador usado internamente por RFE
rfe_estimator = LogisticRegression(
    max_iter=3000,
    class_weight="balanced",
    random_state=RANDOM_STATE
)

N_FEATURES_RFE = min(50, X_train.shape[1])

models_with_rfe = {
    "Logistic Regression + RFE": Pipeline(steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        ("rfe", RFE(estimator=rfe_estimator, n_features_to_select=N_FEATURES_RFE)),
        ("model", LogisticRegression(
            max_iter=3000,
            class_weight="balanced",
            random_state=RANDOM_STATE
        ))
    ]),

    "SVM RBF + RFE": Pipeline(steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        ("rfe", RFE(estimator=rfe_estimator, n_features_to_select=N_FEATURES_RFE)),
        ("model", SVC(
            kernel="rbf",
            probability=True,
            class_weight="balanced",
            random_state=RANDOM_STATE
        ))
    ]),
}

```

```

"Random Forest + RFE": Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("rfe", RFE(estimator=rfe_estimator, n_features_to_select=N_FEATURES_RFE)),
    ("model", RandomForestClassifier(
        n_estimators=300,
        class_weight="balanced",
        random_state=RANDOM_STATE,
        n_jobs=-1
    ))
]),

"XGBoost + RFE": Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("rfe", RFE(estimator=rfe_estimator, n_features_to_select=N_FEATURES_RFE)),
    ("model", XGBClassifier(
        n_estimators=300,
        learning_rate=0.05,
        max_depth=4,
        subsample=0.8,
        colsample_bytree=0.8,
        eval_metric="logloss",
        scale_pos_weight=scale_pos_weight,
        random_state=RANDOM_STATE
    ))
])
}

```

Sin selección de

```

results_rfe = []

for model_name, model in models_with_rfe.items():
    model.fit(X_train, y_train)
    metrics = evaluate_model(model, X_test, y_test, model_name)
    metrics["Experimento"] = "RFE con 50 variables"
    results_rfe.append(metrics)

df_results_rfe = pd.DataFrame(results_rfe)
display(df_results_rfe.sort_values(by="ROC-AUC", ascending=False))

```



```
=====
Logistic Regression + RFE
=====
Matriz de confusion:
[[224  62]
 [ 14  40]]

Reporte de clasificacion:
precision    recall  f1-score   support
```

11. Variables seleccionadas por RFE

Esta sección permite identificar qué variables fueron seleccionadas por RFE para uno de los pipelines.

Esto puede ser útil para la discusión del informe:

```
# Recuperar variables seleccionadas desde el pipeline de Logistic Regression + RFE
rfe_pipeline = models_with_rfe["Logistic Regression + RFE"]

# Asegurarse de que ya está entrenado
selected_mask = rfe_pipeline.named_steps["rfe"].support_
selected_features = X_model.columns[selected_mask].tolist()

print(f"Número de variables seleccionadas: {len(selected_features)}")
print("\nVariables seleccionadas:")
for feature in selected_features:
    print("-", feature)
```

```
precision    recall  f1-score   support
Número de variables seleccionadas: 50
```

```
0 0.91 0.82 0.86 286
Variables seleccionadas:
- AGE
- STENO2_AVG
- IMC_POST
- IMC_AVG
- GB
- SIM_GIPERT
```

Métricas principales:

	Modelo	Accuracy	Precision	Recall/Sensitivity	Specificity	F1-score	ROC-AUC	MCC
0	SVR RBF + RFE	0.782353	0.380952	0.592593	0.818182	0.463768	0.840003	0.348116

```
=====
Random Forest + RFE
=====
```

Matriz de confusion:

```
[[286 103]
 [ 40 106]]
```

Reporte de clasificacion:

```
precision    recall  f1-score   support
- S_AD_ORIT 0.87 1.00 0.93 286
- K_SH_POST 1.00 0.22 0.36 54
- ant_im
- laecm
- imc_m_avg 0.94 0.61 0.65 340
weighted avg 0.89 0.88 0.84 340
- ritm_ecg_p_02
- ritm_ecg_p_04
```

Métricas principales:

	Modelo	Accuracy	Precision	Recall/Sensitivity	Specificity	F1-score	ROC-AUC	MCC
0	Random Forest + RFE	0.876471	1.0	0.222222	1.0	0.363636	0.861888	0.44019

```
=====
XGBoost + RFE
=====
```

Matriz de confusion:

```
[[259  71]
 [ 20  84]]
```

Reporte de clasificacion:

```
precision    recall  f1-score   support
- NA_KB 0 0.93 0.91 0.92 286
- LID_KB 1 0.56 0.63 0.59 54
- NA_R_1_n
- NA_R_2_n
- ANTCR_avg 0.74 0.77 0.75 340
weighted avg 0.87 0.86 0.87 340
- ASP_S_n
```

12. Comparación general de resultados

	Modelo	Accuracy	Precision	Recall/Sensitivity	Specificity	F1-score	ROC-AUC	MCC
0	XGBoost + RFE	0.861765	0.557377	0.62963	0.905594	0.591304	0.863054	0.509857

Se combinan los resultados del experimento sin selección de características y el experimento con RFE.

```

Modelo Accuracy Precision Recall/Sensitivity Specificity F1-score ROC-AUC MCC Experimento

df_results = pd.concat([df_results_no_fs, df_results_rfe], ignore_index=True)

cols_order = [
    "Experimento",
    "Modelo",
    "Accuracy",
    "Precision",
    "Recall/Sensitivity",
    "Specificity",
    "F1-score",
    "ROC-AUC",
    "MCC"
]

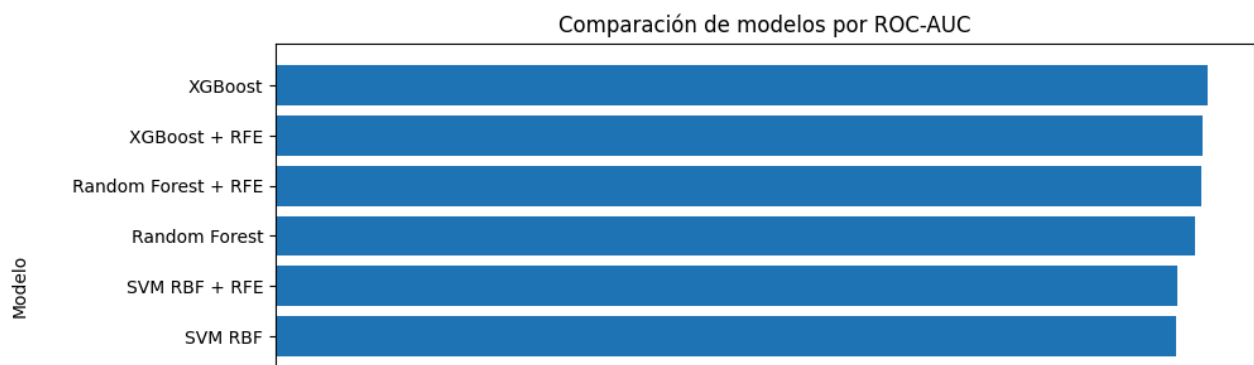
df_results = df_results[cols_order]

display(df_results.sort_values(by="ROC-AUC", ascending=False))

plt.figure(figsize=(10, 5))
plot_data = df_results.sort_values(by="ROC-AUC", ascending=True)
plt.barh(plot_data["Modelo"], plot_data["ROC-AUC"])
plt.title("Comparación de modelos por ROC-AUC")
plt.xlabel("ROC-AUC")
plt.ylabel("Modelo")
plt.show()

```

	Experimento	Modelo	Accuracy	Precision	Recall/Sensitivity	Specificity	F1-score	ROC-AUC	MCC
4	Sin selección de características	XGBoost	0.852941	0.537037	0.537037	0.912587	0.537037	0.868493	0.449624
8	RFE con 50 variables	XGBoost + RFE	0.861765	0.557377	0.629630	0.905594	0.591304	0.863054	0.509857
7	RFE con 50 variables	Random Forest + RFE	0.876471	1.000000	0.222222	1.000000	0.363636	0.861888	0.440190
3	Sin selección de características	Random Forest	0.870588	1.000000	0.185185	1.000000	0.312500	0.856514	0.400617
6	RFE con 50 variables	SVM RBF + RFE	0.782353	0.380952	0.592593	0.818182	0.463768	0.840003	0.348116
2	Sin selección de características	SVM RBF	0.797059	0.384615	0.462963	0.860140	0.420168	0.839355	0.300329
5	RFE con 50 variables	Logistic Regression + RFE	0.776471	0.392157	0.740741	0.783217	0.512821	0.822261	0.417914
1	Sin selección de características	Logistic Regression	0.770588	0.377551	0.685185	0.786713	0.486842	0.810282	0.380809
0	Sin selección de características	Dummy Classifier	0.841176	0.000000	0.000000	1.000000	0.000000	0.500000	0.000000



13. Validación cruzada estratificada

Para una evaluación más robusta, se puede usar validación cruzada estratificada.

Esta sección puede tomar más tiempo en ejecutarse, especialmente con SVM y RFE.

```

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

scoring = {
    "accuracy": "accuracy",
    "precision": "precision",
    "recall": "recall",
    "f1": "f1",
    "roc_auc": "roc_auc",
    "mcc": "mcc"
}

```

```

accuracy : accuracy ,
"precision": "precision",
"recall": "recall",
"f1": "f1",
"roc_auc": "roc_auc",
"mcc": "matthews_corrcoef"
}

cv_models = {
    "Logistic Regression + RFE": models_with_rfe["Logistic Regression + RFE"],
    "Random Forest + RFE": models_with_rfe["Random Forest + RFE"],
    "XGBoost + RFE": models_with_rfe["XGBoost + RFE"]
}

cv_results = []

for model_name, model in cv_models.items():
    print(f"Evaluando con CV: {model_name}")

    scores = cross_validate(
        model,
        X_model,
        y,
        cv=cv,
        scoring=scoring,
        n_jobs=-1,
        return_train_score=False
    )

    row = {"Modelo": model_name}
    for metric_name in scoring.keys():
        row[f"{metric_name}_mean"] = scores[f"test_{metric_name}"].mean()
        row[f"{metric_name}_std"] = scores[f"test_{metric_name}"].std()

    cv_results.append(row)

df_cv_results = pd.DataFrame(cv_results)
display(df_cv_results)

```

Evaluando con CV: Logistic Regression + RFE
 Evaluando con CV: Random Forest + RFE
 Evaluando con CV: XGBoost + RFE

	Modelo	accuracy_mean	accuracy_std	precision_mean	precision_std	recall_mean	recall_std	f1_mean	f1_std	roc_auc
0	Logistic Regression + RFE	0.797647	0.018073	0.418439	0.032025	0.682290	0.072722	0.517516	0.039014	0.
1	Random Forest + RFE	0.872353	0.013491	0.904945	0.072241	0.217239	0.086828	0.343321	0.119994	0.

Pasos siguientes: [New interactive sheet](#)

14. Línea base de literatura

Para el informe, se puede usar como referencia el estudio de Ghafari et al. (2023), que reporta como mejor modelo a XGBoost.

Resultados reportados por ese estudio:

Modelo base reportado	AUC	Sensitivity	Accuracy	F1-score
XGBoost	78.65%	94.35%	91.47%	95.14%

En la discusión preliminar se puede comparar el mejor resultado obtenido en este notebook contra esos valores.

Importante: si los resultados no coinciden exactamente, no significa necesariamente que el código esté mal. Puede deberse a diferencias en partición train/test, imputación, parámetros, implementación de RFE o manejo del desbalance.

```

literature_baseline = pd.DataFrame([
    {
        "Referencia": "Ghafari et al. (2023)",
        "Modelo": "XGBoost + RFE",
        "AUC": 0.7865,
        "Sensitivity": 0.9435,
        "Accuracy": 0.9147,
        "F1-score": 0.9514,
        "MCC": np.nan
    },
    {
        "Referencia": "Newaz et al. (2023)",
        "Modelo": "XGBoost + sampling + cost-sensitive learning",
    }
])

```

```

        "AUC": 0.8088,
        "Sensitivity": np.nan,
        "Accuracy": np.nan,
        "F1-score": np.nan,
        "MCC": 0.6653
    },
    {
        "Referencia": "Khamis et al. (2024)",
        "Modelo": "CatBoost",
        "AUC": 0.9920,
        "Sensitivity": 0.9490,
        "Accuracy": 0.9490,
        "F1-score": np.nan,
        "MCC": 0.8200
    }
])

display(literature_baseline)

best_model_result = df_results.sort_values(by="ROC-AUC", ascending=False).head(1)
display(best_model_result)

```

Referencia			Modelo	AUC	Sensitivity	Accuracy	F1-score	MCC	
0	Ghafari et al. (2023)		XGBoost + RFE	0.7865	0.9435	0.9147	0.9514	NaN	
1	Newaz et al. (2023)	XGBoost + sampling + cost-sensitive learning		0.8088	NaN	NaN	NaN	0.6653	
2	Khamis et al. (2024)		CatBoost	0.9920	0.9490	0.9490	NaN	0.8200	
	Experimento	Modelo	Accuracy	Precision	Recall/Sensitivity	Specificity	F1-score	ROC-AUC	

Pasos siguientes: [New interactive sheet](#)

15. Experimento opcional: balanceo de clases con SMOTE

Esta sección no necesariamente forma parte de la réplica base.

Puede usarse como intento de mejora posterior.

La idea es aplicar SMOTE solo sobre el conjunto de entrenamiento para evitar fuga de información.

```

from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE

models_smote = {
    "Logistic Regression + RFE + SMOTE": ImbPipeline(steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        ("rfe", RFE(estimator=rfe_estimator, n_features_to_select=N_FEATURES_RFE)),
        ("smote", SMOTE(random_state=RANDOM_STATE)),
        ("model", LogisticRegression(
            max_iter=3000,
            random_state=RANDOM_STATE
        ))
    ]),

    "Random Forest + RFE + SMOTE": ImbPipeline(steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        ("rfe", RFE(estimator=rfe_estimator, n_features_to_select=N_FEATURES_RFE)),
        ("smote", SMOTE(random_state=RANDOM_STATE)),
        ("model", RandomForestClassifier(
            n_estimators=300,
            random_state=RANDOM_STATE,
            n_jobs=-1
        ))
    ]),

    "XGBoost + RFE + SMOTE": ImbPipeline(steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        ("rfe", RFE(estimator=rfe_estimator, n_features_to_select=N_FEATURES_RFE)),
        ("smote", SMOTE(random_state=RANDOM_STATE)),
        ("model", XGBClassifier(
            n_estimators=300,
            learning_rate=0.05,
            max_depth=4,
            subsample=0.8,
            colsample_bytree=0.8,

```



```
        eval_metric="logloss",
        random_state=RANDOM_STATE
    ))
    ])
}

results_smote = []

for model_name, model in models_smote.items():
    model.fit(X_train, y_train)
    metrics = evaluate_model(model, X_test, y_test, model_name)
    metrics["Experimento"] = "RFE + SMOTE"
    results_smote.append(metrics)

df_results_smote = pd.DataFrame(results_smote)
display(df_results_smote.sort_values(by="ROC-AUC", ascending=False))
```

```
=====
Logistic Regression + RFE + SMOTE
=====
```

```
Matriz de confusion:
```

```
[[219  67]
```

```
 [13  41]]
```

16. Experimento opcional: CatBoost y LightGBM

Reporte de clasificacion:

El informe plantea CatBoost y LightGBM como modelos candidatos para intentar superar la linea base en fases posteriores.

Esta seccion queda como plantilla opcional. Para la entrega parcial puede dejarse sin ejecutar si el objetivo es solo replicar la linea base de Ghafari et al. (2023).

```
precision recall f1-score support
1 0.38 0.76 0.51 54
accuracy 0.76 340
```

```
# Opcional en Colab:
# !pip install catboost lightgbm -q

# Ejemplo de implementacion posterior:
# from catboost import CatBoostClassifier
# from lightgbm import LGBMClassifier
#
# catboost_model = Pipeline(steps=[
#     ("imputer", SimpleImputer(strategy="median")),
#     ("model", CatBoostClassifier(
#         iterations=300,
#         learning_rate=0.05,
#         depth=4,
#         loss_function="Logloss",
#         eval_metric="AUC",
#         random_seed=RANDOM_STATE,
#         verbose=False
#     ))
# ])
#
# lightgbm_model = Pipeline(steps=[
#     ("imputer", SimpleImputer(strategy="median")),
#     ("model", LGBMClassifier(
#         n_estimators=300,
#         learning_rate=0.05,
#         class_weight="balanced",
#         random_state=RANDOM_STATE
#     ))
# ])
#
# for model_name, model in {
```